

MathLab 3: Module 2

Course Reference: *Data Structures and Algorithms in Python*
Goodrich, Tamassia, Goldwasser (Wiley)

Module 2: Data Structures in Python

Weeks 3–4 Lab: 3 hrs/week Quiz: end of Week 4

Corresponding Chapter(s) in the Book

Chapter 2 – *Object-Oriented Programming*: §2.1 (Design goals – Abstraction, Abstract Data Types).
Chapter 3 – *Algorithm Analysis*: §3.2–3.3 (growth rates, Big-Oh, for complexity intuition).
Chapter 5 – *Array-Based Sequences*: §5.1–5.4 (Python’s sequence types, dynamic arrays, list/tuple/str efficiency).
Chapter 6 – *Stacks, Queues, and Deques*: §6.1–6.3.
Chapter 10 – *Maps, Hash Tables, and Skip Lists*: §10.1 (Maps and Dictionaries), §10.5 (Sets, Multisets, and Multimaps).

Key Concepts – Brief Revision

- Lists are mutable dynamic arrays; tuples are immutable sequences; both support indexing and slicing in $O(1)$ and $O(k)$ respectively.
- Sets store unique unordered elements with average $O(1)$ membership testing (hash-based); dictionaries (maps) store key→value pairs with the same hashing principle.
- An Abstract Data Type (ADT) specifies *what* operations are supported, not *how* they are implemented; the same ADT (e.g., Stack) can have multiple implementations.
- Stack: LIFO – **push**, **pop**, **top**. Queue: FIFO – **enqueue**, **dequeue**. Both can be implemented with arrays (with care for amortized resizing) or linked structures.
- Amortized analysis explains why appending to a Python list is $O(1)$ *amortized*, even though individual append operations occasionally cost $O(n)$ during resizing.
- Big-Oh notation (O), Big-Omega (Ω), and Big-Theta (Θ) describe asymptotic upper, lower, and tight bounds on running time as input size grows.
- Representing sequences (ordered, allow duplicates), multisets/bags (unordered, allow duplicates – via `collections.Counter` or dict-of-counts), and mappings (key-value, unique keys) each fit different problem needs.

Assignment

1. Implement an **ArrayStack** class from scratch (push, pop, top, is_empty, len) backed by a Python list, following the ADT in §6.1.1, without using `list.pop()`'s built-in LIFO convenience directly – manage an explicit size counter.
2. Using only your **ArrayStack** from Q1, write a function that checks whether a string of brackets `(){}[]` is balanced.
3. Implement an **ArrayQueue** class using a circular array (as in §6.2.2), and demonstrate that repeated enqueue/dequeue operations do not require shifting elements.
4. Using two stacks, implement a queue (**enqueue** and **dequeue**) and analyze the amortized cost per operation.
5. Write a function that reverses the elements of a queue using only stack operations (no direct list reversal).
6. Implement a **Deque** class supporting **add_first**, **add_last**, **delete_first**, **delete_last** using a Python list, and compare its performance to `collections.deque` for 100,000 operations using `timeit`.
7. Write a program that uses a stack to evaluate a postfix (Reverse Polish Notation) arithmetic expression, e.g., `"3 4 + 2 *"`.
8. Write a program that converts an infix expression (with parentheses and `+` `-` `*` `/`) to postfix notation using a stack, and then evaluates it with your Q7 solution.
9. Implement a simple text-editor undo mechanism using a stack: support `type(text)`, `undo()`, and `show()` operations.
10. Design a class **Multiset** (bag) supporting **add**, **remove**, **count**, and **distinct_elements**, backed by a dictionary mapping element $\rightarrow$ count. State its ADT explicitly in comments before implementing it.
11. Write a function that, given two lists, returns their multiset intersection (i.e., an element appearing a times in one and b times in the other appears $\min(a, b)$ times in the result).
12. Implement a simple **Map** ADT class (as in §10.1.1) using an unsorted list of key-value pairs, supporting `__getitem__`, `__setitem__`, `__delitem__`, `__len__`. State the worst-case complexity of each operation.
13. Write a program that reads a paragraph of text and uses a Python dictionary to count word frequencies (§10.1.2), then prints the 5 most frequent words.
14. Using two Python dictionaries, implement a simple bidirectional map (each key maps to a value and vice versa), keeping both directions consistent on insertion and deletion.
15. Given two lists representing sets (possibly with duplicates), write functions **union**, **intersection**, and **difference** without using Python's built-in **set** type, then verify your results against the built-in **set** operations.

16. Implement a simple LRU (Least-Recently-Used) cache of fixed capacity using a dictionary plus a list (or `collections.OrderedDict` is NOT allowed) to track recency, supporting `get(key)` and `put(key, value)`.
17. Empirically measure and plot (using SageMath's `list_plot`) the time taken to append n elements to a Python list for $n = 10^3, 10^4, 10^5, 10^6$, and relate the shape of the curve to amortized $O(1)$ append.
18. Empirically compare membership testing (`x in container`) for a Python `list` versus a `set`, for containers of size 10^5 , over 1000 lookups. Report timings and explain the asymptotic difference.
19. Implement a dynamic array class `DynamicArray` from scratch (as in §5.3.1) that manually manages an underlying fixed-size array (using `ctypes` as the book does) and doubles capacity when full. Add print statements to show when resizing occurs.
20. Using your `DynamicArray` from Q19, count the total number of element copies made while inserting $n = 1000$ elements one at a time, and compare it to the theoretical amortized bound of $O(n)$ total copies.
21. Write a function that removes duplicate elements from a list while preserving the original order, using a set to track seen elements, and analyze its time complexity.
22. Given a tuple representing an immutable point `(x, y)`, write a function that returns a new tuple representing the point reflected across the line $y = x$, and explain why tuples (not lists) are appropriate here.
23. Implement a simple 2D matrix class using a list of lists, supporting matrix addition and multiplication, and test it on SageMath matrices for verification (`Matrix(...)`, compare entrywise).
24. Design and implement a `PriorityBag` class (a multiset where each element also has a priority) supporting `insert(item, priority)` and `remove_highest_priority()` using an unsorted list (linear scan for removal).
25. Write a program simulating a hot-potato / Josephus-style elimination game using a queue: given n players and a count k , repeatedly rotate and eliminate every k -th player, printing the elimination order.
26. Given a large text, implement word-frequency counting using (a) a plain dictionary and (b) a list of `(word, count)` pairs with linear search. Time both approaches on a text of 10,000 words and explain the gap.
27. Implement a `Sparse Vector` class using a dictionary (index $\rightarrow$ nonzero value) supporting addition and dot product with another sparse vector, and compare memory usage conceptually to a dense list representation for a vector of length 10^6 with only 10 nonzero entries.
28. Write a function that, given a sequence of stack operations as strings (e.g., `["push 3", "push 5", "pop", ...]`), simulates them on your `ArrayStack` and returns the final stack contents.
29. Implement set operations (union, intersection, symmetric difference) using Python dictionaries as your only underlying structure (no built-in `set`), and prove via testing that your `symmetric_difference` is commutative on 10 random list pairs.

30. Design a small ADT-driven module: define an abstract base class `Container` (using `abc.ABC`) with abstract methods `add`, `remove`, `__contains__`; then implement two concrete subclasses, `ListContainer` and `SetContainer`, and write a test harness that runs identical operations on both and checks consistent behavior.